

The background of the image features a dark blue gradient on the left, transitioning into a large, vibrant, abstract shape on the right. This shape is composed of overlapping curved segments in shades of orange, pink, and purple, creating a dynamic, modern aesthetic.

AWS re:Invent

NOV. 27 – DEC. 1, 2023 | LAS VEGAS, NV

OPN312

How Amazon ECR speaks open source: OCI and the v2 Proxy

Matt Kang

Sr. SDE
Amazon ECR

Adam Long

SDE
Amazon ECR



It starts with a push



More use cases

- Build delivery
- Scaled distribution
- Workload deployments
- Content discovery
- Inventory auditing
- Lifecycle management
- Migrations and backups



More tools



What is the OCI?

Open Container Initiative was formed in 2015 to define standards for all these tools to interoperate

Specifications and related tools for runtime, image, and distribution



<https://opencontainers.org/>

For context: What is an image?

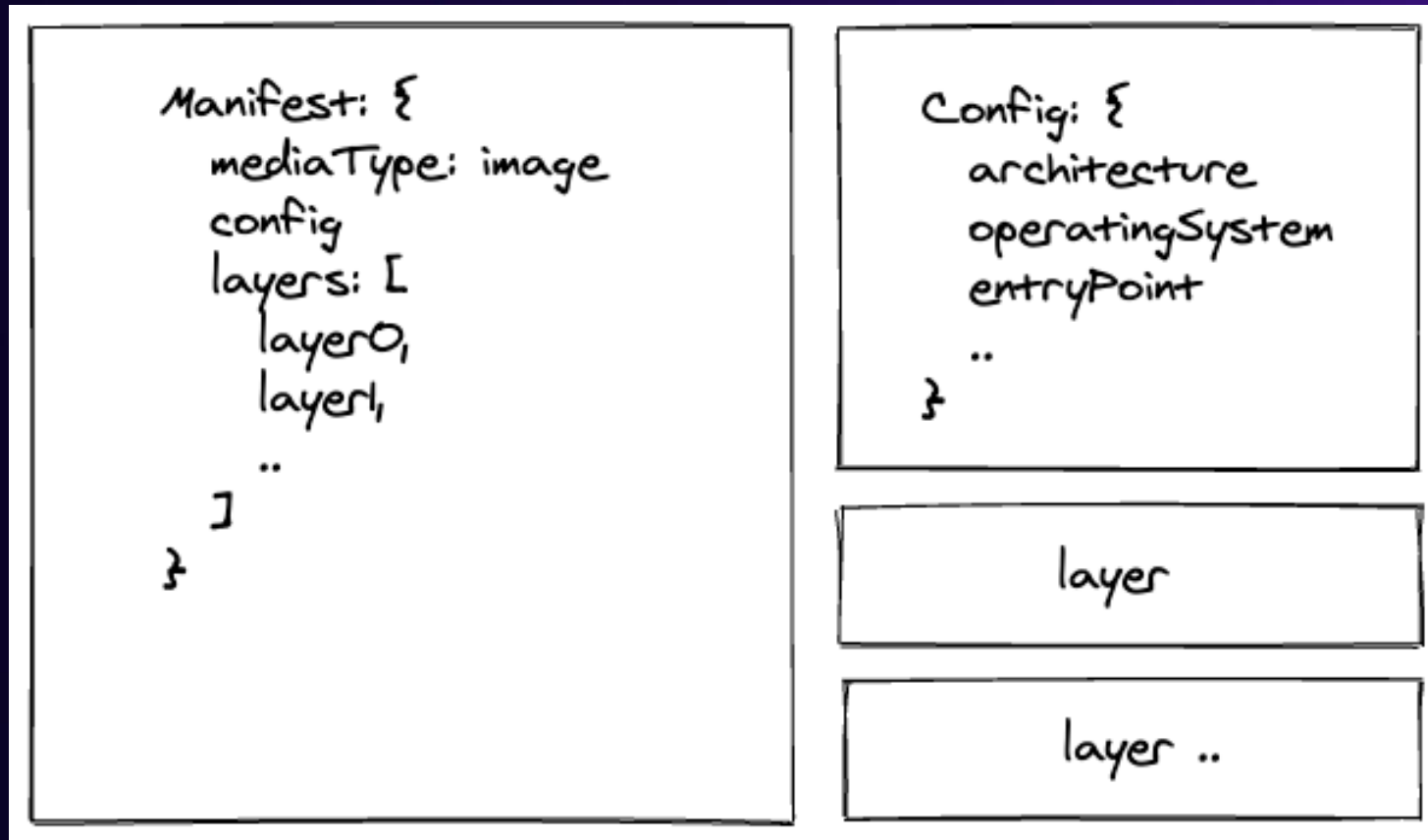
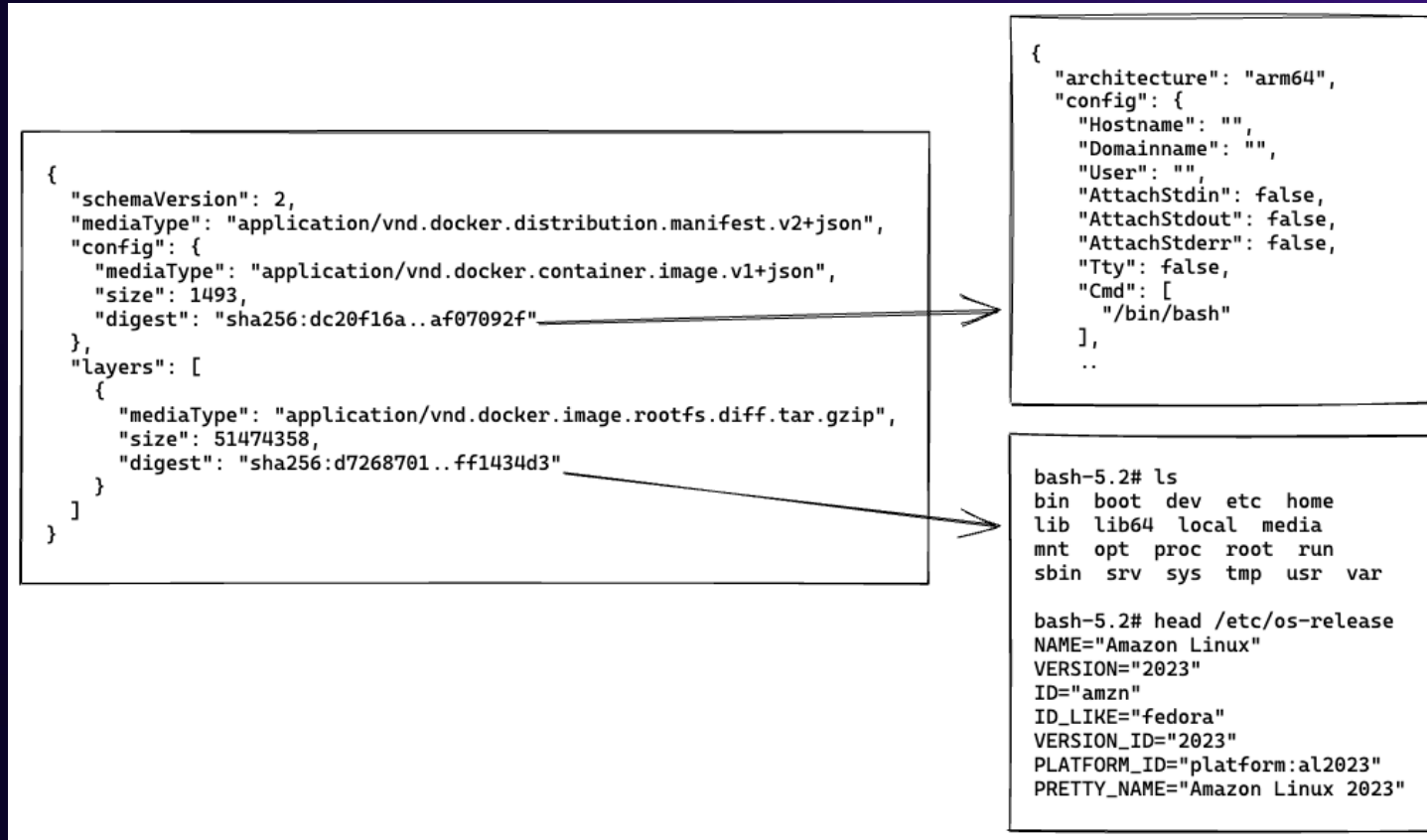


Image manifest, config, and layers



manifest

blob(s)

What about multiplatform images?

```
Manifest: {  
  mediaType: index  
  config  
  manifests: [  
    manifest0,  
    manifest1,  
    ..  
  ]  
}
```

manifest

```
Manifest: {  
  mediaType: image  
  config  
  layers: [  
    layer0,  
    layer1,  
    ..  
  ]  
}
```

manifest(s)

```
Config: {  
  architecture  
  operatingSystem  
  entryPoint  
  ..  
}
```

layer

layer ..

blob(s)

Manifest lists, or the OCI Index

```
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.docker.distribution.manifest.list.v2+json",
  "manifests": [
    {
      "mediaType": "application/vnd.docker.distribution.manifest.v2+json",
      "size": 529,
      "digest": "sha256:72e295c1..aadealcfc",
      "platform": {
        "architecture": "arm64",
        "os": "linux",
        "variant": "v8"
      }
    },
    ..
  ]
}
```

manifest

```
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.docker.distribution.manifest.v2+json",
  "config": {
    "mediaType": "application/vnd.docker.container.image.v1+json",
    "size": 1493,
    "digest": "sha256:dc20f16a..af07092f"
  },
  "layers": [
    {
      "mediaType": "application/vnd.docker.image.rootfs.diff.tar.gzip",
      "size": 51474358,
      "digest": "sha256:d7268701..ff1434d3"
    }
  ]
}
```

manifest(s)

```
{
  "architecture": "arm64",
  "config": {
    "Hostname": "",
    "Domainname": "",
    "User": "",
    "AttachStdin": false,
    "AttachStdout": false,
    "AttachStderr": false,
    "Tty": false,
    "Cmd": [
      "/bin/bash"
    ],
    ..
  }
}
```

```
bash-5.2# ls
bin boot dev etc home
lib lib64 local media
mnt opt proc root run
sbin srv sys tmp usr var

bash-5.2# head /etc/os-release
NAME="Amazon Linux"
VERSION="2023"
ID="amzn"
ID_LIKE="fedora"
VERSION_ID="2023"
PLATFORM_ID="platform:al2023"
PRETTY_NAME="Amazon Linux 2023"
```

blob(s)

It's manifests and blobs all the way down

Everything stored in a registry is a manifest or a blob

All objects are content-addressable, identified by digest

Manifests contain metadata and content descriptors

Content descriptors describe blobs and point to them

OCI distribution endpoints

OCI distribution specifies a set of HTTP endpoints for manipulating registry content

Pushing images

```
POST /v2/<name>/blobs/uploads/  
PUT /v2/<name>/blobs/uploads/<reference>  
PATCH /v2/<name>/blobs/uploads/<reference>  
PUT /v2/<name>/manifests/<reference>
```

Pulling images

```
GET /v2/<name>/manifests/<reference>  
GET /v2/<name>/blobs/<digest>
```

Content discovery

```
HEAD /v2/<name>/manifests/<reference>  
HEAD /v2/<name>/blobs/<digest>  
GET /v2/<name>/tags/list
```

Coming soon...

```
GET /v2/<name>/referrers/<digest>
```

Amazon ECR APIs

Amazon ECR has APIs for pushing and pulling content

Pushing images

ecr:PutImage
ecr:InitiateLayerUpload
ecr:UploadLayerPart

Pulling images

ecr:BatchGetImage
ecr:GetDownloadUrlForLayer

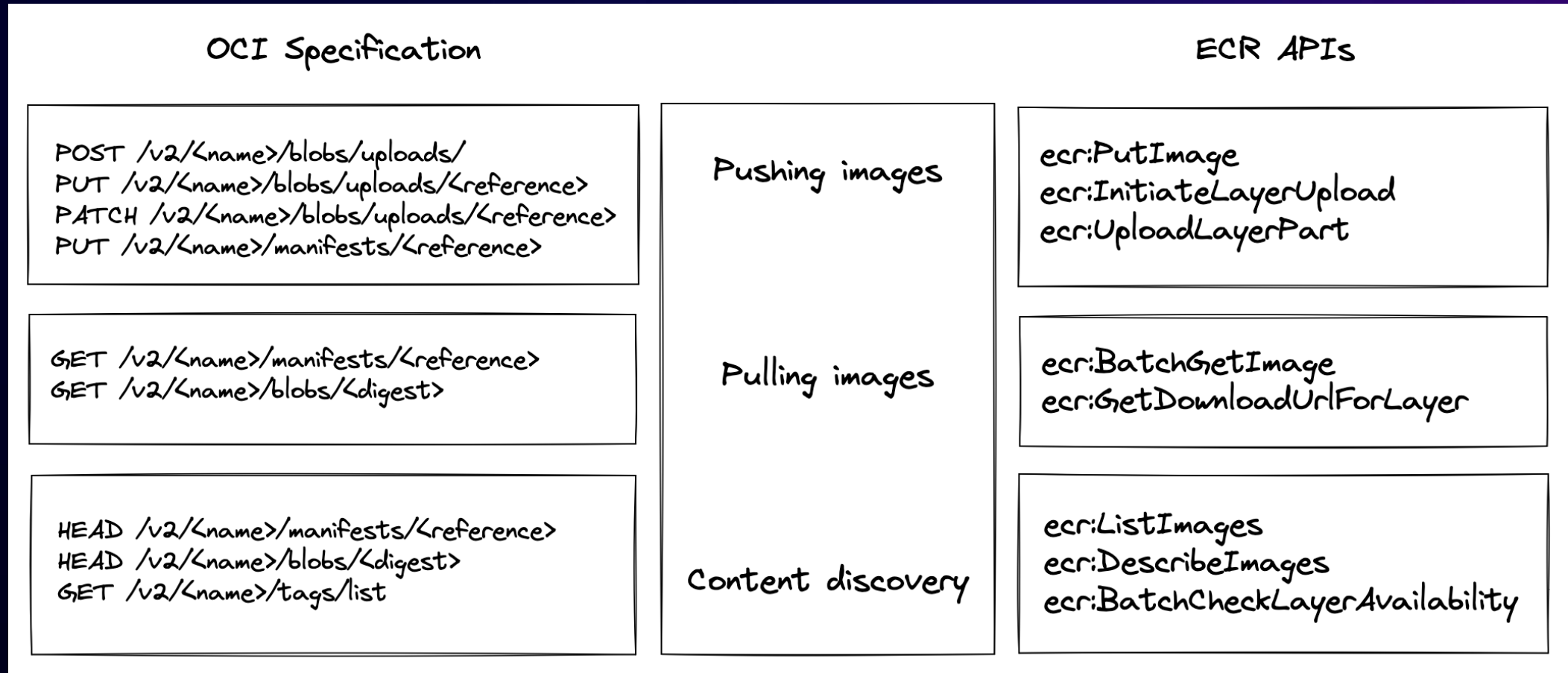
Content discovery

ecr:ListImages
ecr:DescribeImages
ecr:BatchCheckLayerAvailability

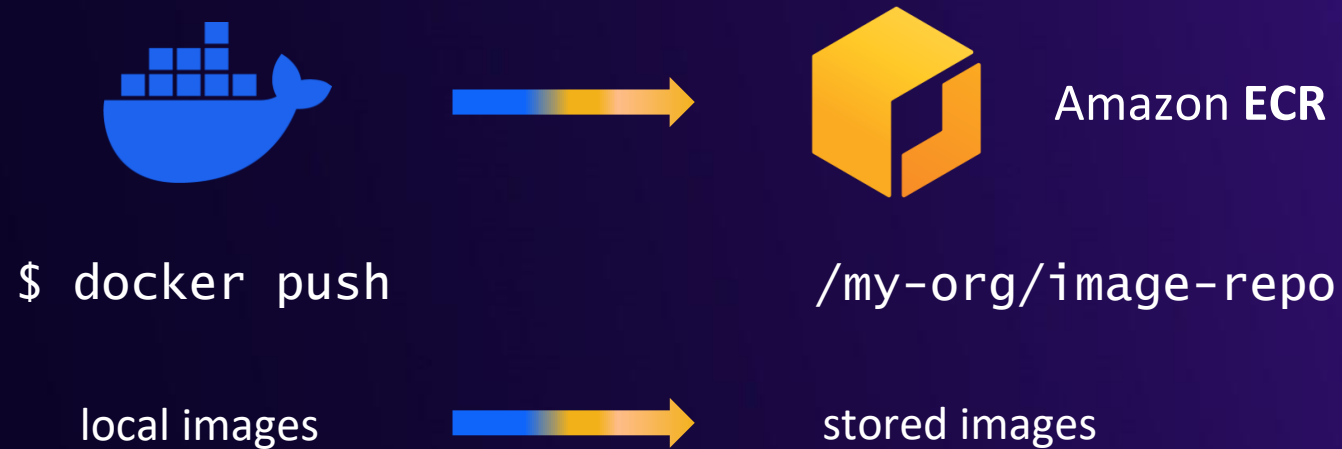
Other operations

additional ECR APIs...

Bridging the gap: v2 proxy



So back to that push



So back to that push



Amazon ECR

\$ docker push

/my-org/image-repo

POST layer0



/v2/my-org/image-repo/blobs/uploads

POST layer1..



/v2/my-org/image-repo/blobs/uploads

PUT manifest



/v2/my-org/image-repo/manifests/<reference>

How about a pull



Amazon ECR

`$ docker pull`

`/my-org/image-repo`

`GET manifest`



`/v2/my-org/image-repo/manifests/<ref>`

`GET layer0`



`/v2/my-org/image-repo/blobs/<digest>`

`GET layer1..`



`/v2/my-org/image-repo/blobs/<digest>`

Content discovery



checkImages

GET tags

GET manifest



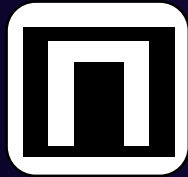
Amazon ECR

/my-org/image-repo

/v2/my-org/image-repo/tags/list

/v2/my-org/image-repo/manifests/<digest>

Referrers (coming soon)



verifySignature

GET tags

GET manifest

GET signature(s)



Amazon ECR

/my-org/image-repo

/v2/my-org/image-repo/referrers/<digest>

/v2/my-org/image-repo/manifests/<digest>

/v2/my-org/image-repo/blobs/<digest>

Client code

go-containerregistry is widely adopted

 github.com/google/go-containerregistry

oras-go is geared toward artifact management

 github.com/oras-project/oras-go

Python and Rust have some support, the specs are open
and implementer-friendly

Additional questions from the room?

Thank you!



Please complete the session survey in the mobile app

Matt Kang

X @kattmang

in linkedin.com/in/mattkangcodes

Adam Long

X @alpeople

in linkedin.com/in/adam-long

